

Generador de Logs Crudos Masivos

Reporte técnico de benchmarking — Fase 1

Caso Terminal Puerto Coquimbo (TPC)

Autor(es): Bernardo Casanga, Boris Berrios, Bruce Inostroza, Eduardo Manzana, Maximiliano Uribe, Esteban Cortés

Fecha: 8 de septiembre de 2026

Síntesis

Se generó un dataset sintético de 10.000.004 eventos de sensores IoT portuarios en formato JSON Lines (1.289.499.073 bytes) y se midió la escalabilidad de dos arquitecturas de escritura concurrente —escritor único con cola (queue) y archivos segmentados por proceso (shard)— con 1, 2, 4, 8 y 16 trabajadores, manteniendo fijo el total de eventos. El speedup máximo observado fue 5.03x con 16 trabajadores sobre un equipo de 8 núcleos físicos y 16 hilos lógicos, con una eficiencia de 31.4 %. El punto dulce, definido como la última configuración con eficiencia mayor o igual a 50 %, es 4 trabajadores (69.9 %), que es la configuración con la que se generó el dataset final de producción. El análisis identifica una fracción no paralelizable estable de aproximadamente 0.146 como causa dominante del aplanamiento, y descarta la saturación de disco con evidencia propia.

1. Descripción del entorno de pruebas

Todas las mediciones de este informe se obtuvieron en un único equipo, sin cambios de configuración entre corridas y sin otras cargas de trabajo activas más allá de las del sistema operativo. La Tabla 1 documenta la plataforma con el detalle necesario para reproducir el experimento.

Tabla 1. Plataforma de pruebas.

Componente	Especificación
Procesador	AMD Ryzen 7 4800H
Núcleos físicos	8
Núcleos lógicos (SMT)	16
Memoria RAM	16 GB DDR4
Disco	NVMe SSD Samsung MZVLQ512HALU-000H1
Sistema operativo	Windows 11 Home Single Language
Versión de Python	3.14.4

1.1 Consecuencias de esta plataforma sobre el diseño experimental

Tres características del equipo condicionan directamente la interpretación de los resultados y conviene fijarlas antes de mostrar cualquier número:

- **La relación 8 físicos / 16 lógicos define una frontera dura en $n = 8$.** Las configuraciones de 1 a 8 trabajadores reparten trabajo entre núcleos de ejecución independientes; la de 16 reparte entre hilos SMT que comparten las unidades funcionales de un mismo núcleo. Cualquier lectura del tramo 8 → 16 debe hacerse sabiendo que allí no se agregó hardware de cómputo real, solo contextos de ejecución.
- **El disco es NVMe, no mecánico.** Esto vuelve la hipótesis de saturación de disco menos probable a priori, pero no la descarta: en la sección 7 se descarta con mediciones propias, no con la ficha técnica del dispositivo. El ancho de banda máximo real del disco no se midió (dato no disponible).
- **Windows obliga a usar el método de arranque `spawn` de multiprocessing.** Cada proceso hijo vuelve a importar el módulo y reconstruye su intérprete, a diferencia de `fork` en Linux. Ese costo se paga una vez por proceso y crece linealmente con n ; su magnitud se acota en la sección 7.

1.2 Qué se cronometró exactamente

El tiempo de cada corrida no es el tiempo del subprocesso completo, sino el valor `duracion_seg` que escribe el manifiesto, medido con `time.perf_counter()` en `src/main.py:104` y `src/main.py:166`, es decir alrededor de la generación pura. Queda deliberadamente fuera de la ventana medida el arranque e importación del intérprete de Python del proceso padre y la validación línea por línea del dataset que `src/main.py` ejecuta después (líneas 179-194). El arnés de benchmarking lee ese valor del manifiesto en `bench/benchmark.py:44`, de modo que todas las configuraciones se cronometran con el mismo criterio. La creación de los procesos hijos sí queda dentro de la ventana, porque ocurre después de `t_inicio`; esto es intencional, ya que el costo de `spawn` forma parte del costo real de paralelizar.

2. Curva de escalabilidad con carga fija

El experimento mantiene constante el trabajo total —10.000.004 eventos por corrida, semilla 42— y varía únicamente el número de trabajadores en 1, 2, 4, 8 y 16. Se trata, por lo tanto, de un escalamiento fuerte (strong scaling): el problema no crece con los recursos, de modo que cualquier tiempo que no baja proporcionalmente al agregar procesos revela una limitación real y no un aumento encubierto de la carga.

2.1 Protocolo de medición

- Cada configuración se ejecuta cuatro veces. La repetición 1 se marca como `warmup` y se descarta explícitamente del cálculo; las repeticiones 2, 3 y 4 son las tres mediciones válidas exigidas.

- Se reporta la **mediana** de esas tres mediciones, no la mejor corrida. Con tres valores, la mediana es el valor central, lo que la vuelve inmune al outlier aislado que produce cualquier interferencia del sistema operativo.

El directorio de salida se vacía antes de cada repetición ([bench/benchmark.py:93-96](#)), de modo que ninguna corrida se beneficia de archivos ya presentes, y el dataset final en [data/raw](#) nunca se toca: el benchmark escribe siempre en un directorio temporal.

Tabla 2. Corridas completas de la arquitectura shard (20 corridas). La repetición 1 de cada bloque es la corrida de calentamiento, descartada.

Work.	Rep.	Tiempo (s)	eventos/s	MB/s	bytes	Tipo
1	1	91.363	109454	13.46	1289604087	warmup
1	2	90.506	110490	13.59	1289604087	medicion
1	3	90.491	110508	13.59	1289604087	medicion
1	4	90.463	110542	13.60	1289604087	medicion
2	1	46.456	215258	26.47	1289570202	warmup
2	2	46.608	214556	26.39	1289570202	medicion
2	3	47.927	208651	25.66	1289570202	medicion
2	4	47.646	209881	25.81	1289570202	medicion
4	1	31.157	320955	39.47	1289499073	warmup
4	2	33.077	302325	37.18	1289499073	medicion
4	3	32.362	309005	38.00	1289499073	medicion
4	4	30.643	326339	40.13	1289499073	medicion
8	1	22.628	441931	54.35	1289556872	warmup
8	2	22.879	437082	53.75	1289556872	medicion
8	3	23.195	431128	53.02	1289556872	medicion
8	4	22.342	447588	55.05	1289556872	medicion
16	1	19.844	503931	62.15	1293307357	warmup
16	2	18.001	555525	68.52	1293307357	medicion
16	3	17.804	561672	69.28	1293307357	medicion
16	4	18.235	548396	67.64	1293307357	medicion

Tabla 3. Corridas completas de la arquitectura queue, medidas en las mismas condiciones.

Work.	Rep.	Tiempo (s)	eventos/s	MB/s	bytes	Tipo
1	1	94.297	106048	13.04	1289604087	warmup
1	2	92.477	108135	13.30	1289604087	medicion
1	3	93.550	106895	13.15	1289604087	medicion
1	4	94.050	106326	13.08	1289604087	medicion
2	1	49.400	202429	24.90	1289570202	warmup
2	2	50.108	199569	24.54	1289570202	medicion
2	3	52.997	188690	23.21	1289570202	medicion
2	4	52.732	189638	23.32	1289570202	medicion
4	1	32.569	307041	37.76	1289499073	warmup
4	2	31.226	320246	39.38	1289499073	medicion
4	3	31.407	318400	39.16	1289499073	medicion
4	4	30.486	328020	40.34	1289499073	medicion
8	1	23.139	432171	53.15	1289556872	warmup
8	2	22.770	439175	54.01	1289556872	medicion
8	3	22.758	439406	54.04	1289556872	medicion
8	4	23.497	425586	52.34	1289556872	medicion
16	1	19.105	523423	64.56	1293307357	warmup
16	2	18.900	529101	65.26	1293307357	medicion
16	3	18.978	526926	64.99	1293307357	medicion
16	4	18.926	528374	65.17	1293307357	medicion

2.2 Medianas por configuración

Tabla 4. Tiempo mediano de las tres mediciones válidas, por arquitectura.

Trabajadores	T mediana shard (s)	eventos/s shard	T mediana queue (s)	eventos/s queue
1	90.491	110508	93.550	106895
2	47.646	209881	52.732	189638
4	32.362	309005	31.226	320246
8	22.879	437082	22.770	439175
16	18.001	555525	18.926	528374

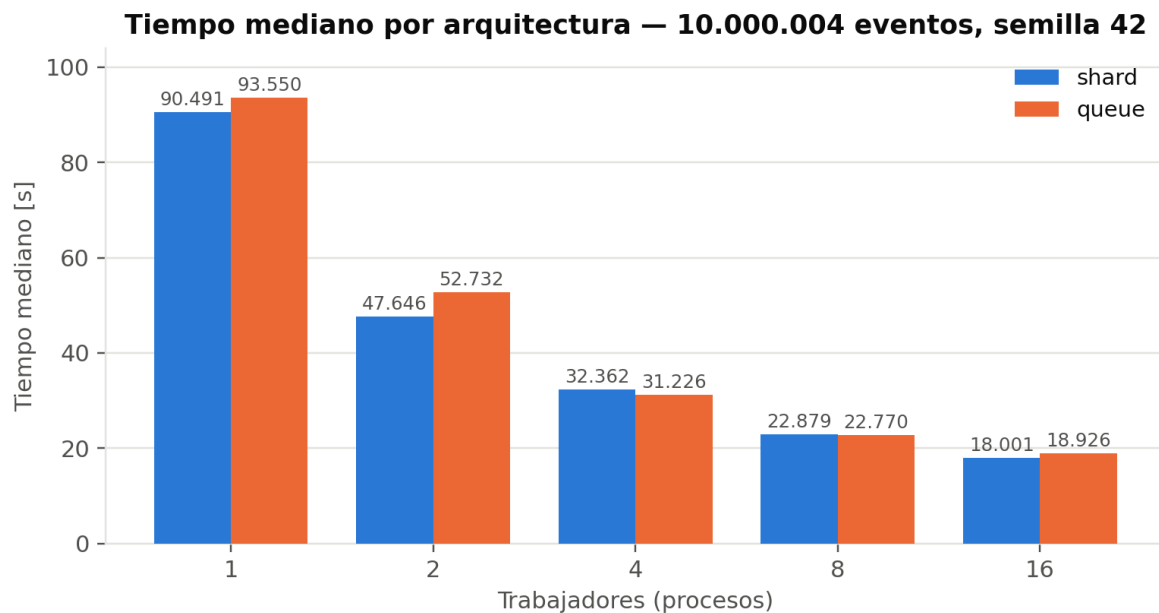


Figura 1. Tiempo mediano por configuración y arquitectura.

2.3 Dispersión y honestidad sobre la corrida de calentamiento

La dispersión entre las tres mediciones válidas es baja pero no despreciable, y no es uniforme: 0.05 % con 1 trabajador (90.463 s a 90.506 s), 2.83 % con 2, 7.94 % con 4 (30.643 s a 33.077 s), 3.82 % con 8 y 2.42 % con 16. La configuración de 4 trabajadores es la más ruidosa del conjunto, lo que refuerza la decisión de reportar la mediana en lugar del promedio o del mejor tiempo.

Corresponde señalar algo que las mediciones muestran y que sería cómodo omitir: la corrida de calentamiento no fue sistemáticamente más lenta. En tres de las cinco configuraciones de shard (2, 4 y 8 trabajadores) el warmup fue más rápido que la mediana de las mediciones posteriores. Descartarlo sigue siendo correcto como decisión metodológica tomada de antemano —evita que el estado inicial de las cachés del sistema de archivos contamine la comparación—, pero en este equipo no está justificado a posteriori por los datos: no hay un efecto de calentamiento visible. Se reporta así en lugar de presentar el descarte como si los números lo respaldaran.

3. Speedup y eficiencia frente al ideal

Sobre las medianas de la Tabla 4 se calculan las dos métricas centrales del informe: el speedup $S(n) = T(1) / T(n)$, que mide cuántas veces más rápido corre el programa con n trabajadores, y la eficiencia $E(n) = S(n) / n$, que mide qué fracción de cada trabajador agregado se convierte efectivamente en trabajo útil. El speedup ideal es $S(n) = n$ y la eficiencia ideal es 100 %.

Tabla 5. Speedup y eficiencia de la arquitectura shard, y brecha absoluta respecto del ideal.

Trabajadores	T mediana (s)	$S(n)$	$E(n)$	S ideal	Brecha (ideal – medido)
1	90.491	1.00x	100.0 %	1x	0.00

Trabajadores	T mediana (s)	S(n)	E(n)	S ideal	Brecha (ideal – medido)
2	47.646	1.90x	95.0 %	2x	0.10
4	32.362	2.80x	69.9 %	4x	1.20
8	22.879	3.96x	49.4 %	8x	4.04
16	18.001	5.03x	31.4 %	16x	10.97

Tabla 6. Speedup y eficiencia de la arquitectura queue, con las mismas fórmulas.

Trabajadores	T mediana (s)	S(n)	E(n)	Brecha (ideal – medido)
1	93.550	1.00x	100.0 %	0.00
2	52.732	1.77x	88.7 %	0.23
4	31.226	3.00x	74.9 %	1.00
8	22.770	4.11x	51.4 %	3.89
16	18.926	4.94x	30.9 %	11.06

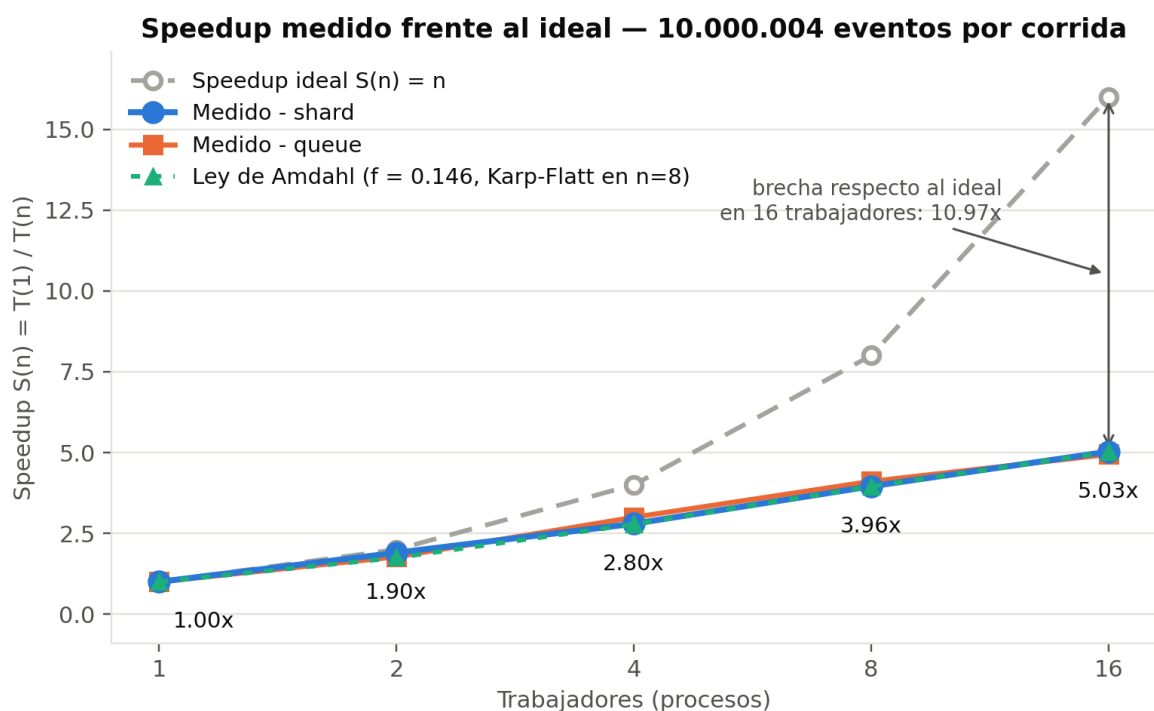


Figura 2. Speedup medido frente a la recta de speedup ideal $S(n) = n$. La curva a trazos verde es el modelo de Amdahl ajustado con la fracción serial estimada en $n = 8$ (sección 7); cae prácticamente sobre lo medido.

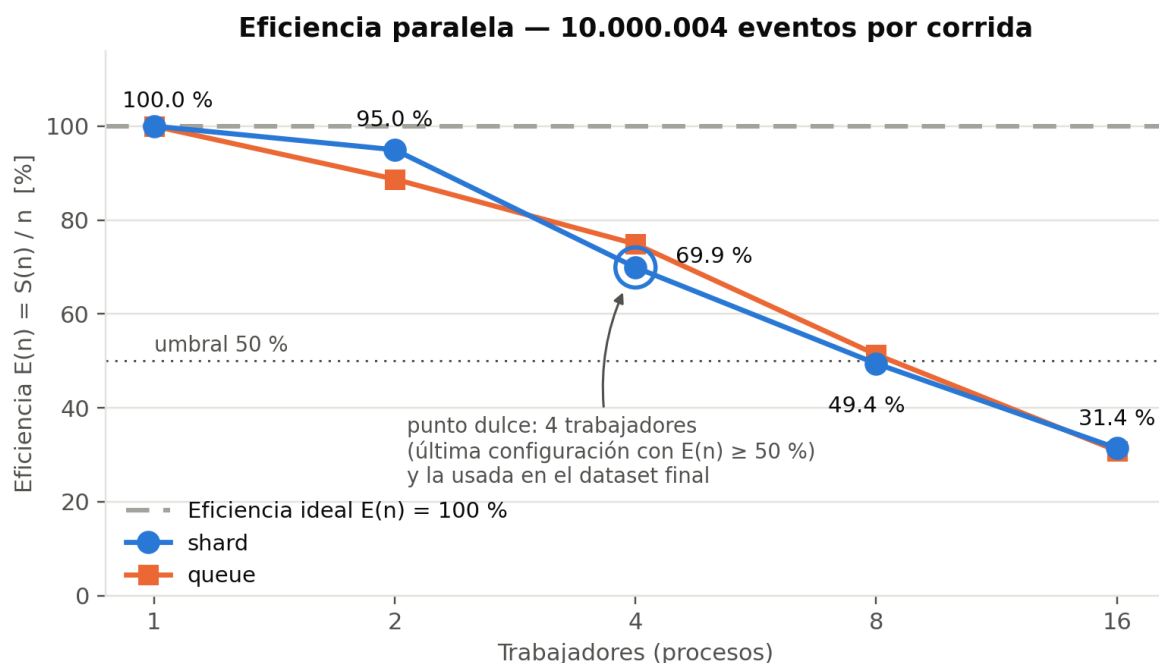


Figura 3. Eficiencia paralela. El umbral de 50 % define el punto dulce; se cruza entre 4 y 8 trabajadores.

3.1 Lectura de la distancia entre las dos curvas

La distancia entre la curva medida y la recta ideal es el resultado principal de este informe, y su forma dice más que su tamaño. Con 2 trabajadores la brecha es de apenas 0.10 (95.0 % de eficiencia): el sistema paraleliza casi perfectamente. A partir de ahí la brecha se ensancha de forma acelerada: 1.20 con 4 trabajadores, 4.04 con 8 y 10.97 con 16. Es decir, con 16 procesos el equipo entrega menos de un tercio del rendimiento que entregaría un paralelismo perfecto.

La medida más informativa no es la brecha acumulada sino la ganancia marginal por trabajador agregado, que se obtiene dividiendo el incremento de speedup entre el número de trabajadores incorporados. Al pasar de 2 a 4 trabajadores, cada uno de los dos nuevos procesos aporta 0.448 de speedup; de 4 a 8, cada uno de los cuatro nuevos aporta 0.290; de 8 a 16, cada uno de los ocho nuevos aporta 0.134. El rendimiento marginal se reduce aproximadamente a la mitad en cada duplicación. Esa caída regular, y no un colapso brusco en un punto concreto, es la forma real del aplanamiento en este sistema.

Ambas arquitecturas describen esencialmente la misma curva, lo que ya adelanta una conclusión: el factor que limita la escalabilidad no está en el mecanismo de escritura, porque cambiarlo por completo no cambia la forma de la curva. Sí hay diferencias de detalle que se analizan en la sección 7.

4. Rendimiento de entrada/salida

El rendimiento se expresa en dos unidades complementarias: eventos por segundo, que mide el trabajo lógico del generador, y MB/s efectivamente escritos, que mide el caudal real hacia el disco. Ambos derivan del mismo tiempo mediano, de modo que describen el mismo fenómeno en escalas distintas; se reportan los dos porque el enunciado los exige y porque su relación permite verificar la consistencia interna de los datos.

Tabla 7. Rendimiento de E/S por configuración (medianas de las tres mediciones válidas).

Work.	eventos/s shard	MB/s shard	eventos/s queue	MB/s queue	bytes escritos
1	110508	13.59	106895	13.15	1289604087
2	209881	25.81	189638	23.32	1289570202
4	309005	38.00	320246	39.38	1289499073
8	437082	53.75	439175	54.01	1289556872
16	555525	68.52	528374	65.17	1293307357

Rendimiento de E/S por configuración — 10.000.004 eventos

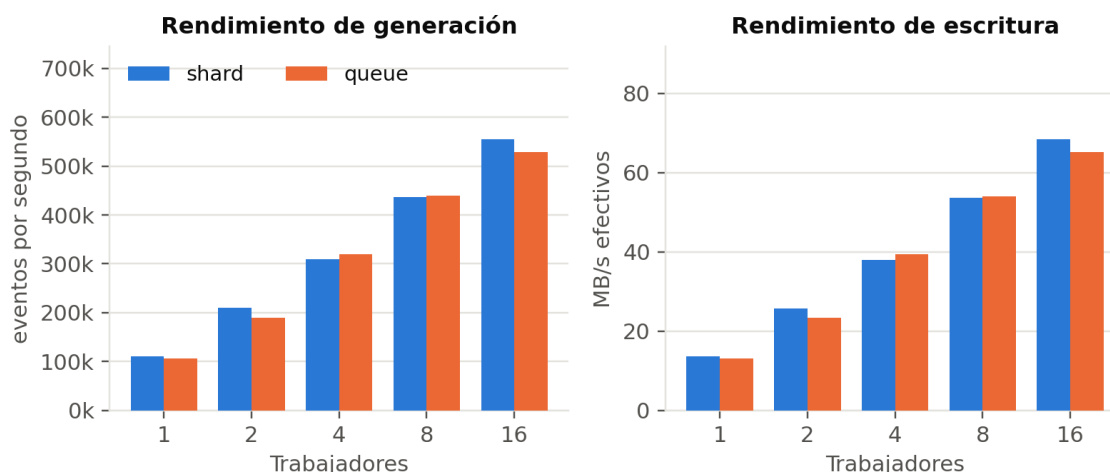


Figura 4. Rendimiento de generación y de escritura por configuración. Las dos medidas se grafican en paneles separados por tener unidades distintas.

4.1 Observaciones sobre el volumen escrito

El tamaño total del dataset no es idéntico entre configuraciones, y la diferencia es informativa. Las configuraciones de 1 a 8 trabajadores producen entre 1.289.499.073 y 1.289.604.087 bytes —una variación de 105.014 bytes, menos del 0.01 %—, mientras que la de 16 trabajadores produce 1.293.307.357 bytes, es decir 3.750.485 bytes más que la de 8.

Esa diferencia mayor tiene una explicación aritmética verificable en el esquema del evento. El campo `worker_id` se serializa como entero: con 16 trabajadores, los identificadores 10 a 15 ocupan dos caracteres en lugar de uno, lo que agrega exactamente un byte por evento a 6 de cada 16 trabajadores. Sobre 10.000.004 eventos eso predice 3.750.002 bytes adicionales, frente a los 3.750.485 observados. La coincidencia hasta el cuarto dígito significativo confirma el mecanismo. Las diferencias menores entre las configuraciones de 1 a 8 se atribuyen a que cada trabajador usa su propio flujo pseudoaleatorio `random.Random(semilla + worker_id)`, por lo que un reparto distinto genera valores numéricos de longitud textual distinta; esta atribución es una interpretación coherente con el código y no una medición directa.

El caudal de escritura crece de forma monótona en todo el rango, de 13.59 MB/s con un trabajador a 68.52 MB/s con dieciséis, sin llegar en ningún momento a una meseta. Este hecho se retoma en la sección 7, porque es la primera evidencia contra la hipótesis de saturación de disco.

5. Costo de pickle en la arquitectura queue

En la arquitectura de escritor único, todo dato que va de un trabajador al escritor cruza una `multiprocessing.Queue`, y ese cruce implica serializar el objeto con `pickle`, escribirlo en una tubería del sistema operativo y deserializarlo al otro lado. El costo tiene dos componentes de naturaleza distinta: uno proporcional al volumen de datos y otro fijo por cada llamada a `put()`. Este experimento aísla el segundo variando el tamaño de lote y dejando todo lo demás constante.

5.1 Dónde se paga exactamente

- `src/arquitectura_a.py:73` — `cola.put((wid, tick0, buf))` dentro del bucle del trabajador: aquí se serializa la lista completa de líneas del lote. Es el punto donde se paga el costo en el lado emisor.
- `src/arquitectura_a.py:78` — el mismo `put()` para el lote residual que queda al terminar los ticks.
- `src/arquitectura_a.py:32` — `lote = cola.get()` en el proceso escritor: aquí se paga la deserialización. Es el lado receptor del mismo costo.

- `src/arquitectura_a.py:72` — `if len(buf) >= 10_000:` es el umbral que fija el tamaño de lote en producción, y por lo tanto la variable que este experimento manipula.

En el arnés de medición, el mismo punto está parametrizado en `bench/costo_pickle.py:65-66`, con el `get()` correspondiente en la línea 32 y la lista de tamaños de lote en la línea 141. La arquitectura shard, por contraste, no paga nada de esto: cada trabajador escribe directamente a su propio descriptor de archivo (`src/arquitectura_b.py:29` y `src/arquitectura_b.py:34`), y lo único que cruza un `pickle` es el diccionario de resultado que devuelve al `Pool`, una sola vez por trabajador.

5.2 Resultados

Condiciones del experimento: 10.000.004 eventos, 4 trabajadores fijos, arquitectura queue, cola con `maxsize = 64`. Solo varía el número de eventos por `put()`.

Tabla 8. Costo de pickle por tamaño de lote. El número de llamadas a `put()` es un valor derivado (eventos ÷ tamaño de lote).

Eventos por put()	Duración (s)	eventos/s	Llamadas a put() (aprox.)	µs por evento	Sobrecosto vs lote 1.000
1	55.423	180430	10.000.004	5.542	+40.11 %
100	41.172	242884	100.000	4.117	+4.08 %
1.000	39.558	252794	10.000	3.956	—
10.000	39.806	251218	1.000	3.981	+0.63 %

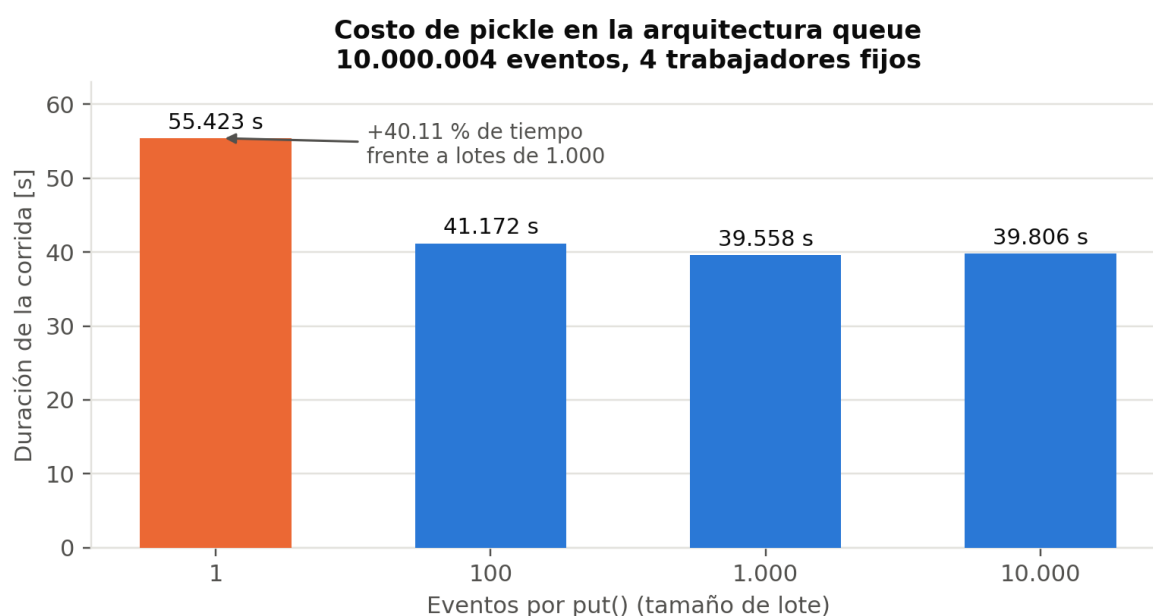


Figura 5. Duración de la corrida según el tamaño de lote. El tamaño en bytes escritos por corrida de este experimento no quedó registrado: dato no disponible.

5.3 Interpretación

El costo por llamada domina por completo el extremo izquierdo de la curva. Enviar los eventos de uno en uno cuesta 55.423 s frente a 39.558 s con lotes de mil: un 40.11 % de tiempo adicional, 15.865 s, para escribir exactamente los mismos bytes en el mismo disco. Repartido sobre las aproximadamente diez millones de llamadas adicionales a `put()`, eso da del orden de 1.59 µs de sobrecosto por llamada, que es el precio combinado de serializar un objeto minúsculo, tomar el candado interno de la cola, escribir en la tubería y volver a deserializar al otro lado.

La curva satura rápido. Con lotes de 100 el sobrecosto ya bajó a 4.08 %, y con lotes de 1.000 se agotó: pasar a 10.000 no mejora nada (39.806 s frente a 39.558 s, un 0.63 % más lento). Corresponde ser explícito sobre el alcance de esa última comparación: a diferencia del experimento de la sección 2, este se ejecutó una sola vez por tamaño

de lote, sin corrida de calentamiento ni repeticiones, de modo que una diferencia de 0.25 s sobre casi 40 s no es distinguible del ruido de medición. Lo que la tabla sostiene con solidez es que el beneficio del loteo se agota entre 100 y 1.000 eventos por put(); lo que no sostiene es que 1.000 sea mejor que 10.000.

El código de producción usa lotes de 10.000, es decir opera en la zona plana de la curva. Esa elección no cuesta rendimiento medible y, como se ve en la sección 6, mantiene acotado el consumo de memoria del escritor por la vía del maxsize de la cola.

6. Gestión de memoria

El generador produce un dataset de 1.229,8 MB, más de setenta veces la memoria que llega a usar el proceso que lo escribe. Que eso sea posible es consecuencia directa de la contrapresión de la cola. La medición se hizo sobre el proceso escritor de la arquitectura queue, instrumentado en `bench/worker_mem.py`: el pico de conjunto residente se obtiene con `GetProcessMemoryInfo` vía ctypes en Windows (líneas 30-64) y el pico de asignaciones del intérprete con `tracemalloc` (líneas 89 y 114).

Tabla 9. Consumo máximo del proceso escritor. Configuración: 10.000.004 eventos, 4 trabajadores, lotes de 10.000, arquitectura queue.

Métrica	Bytes	MB	Qué mide
Pico de conjunto residente (RSS)	47.554.560	45.35	memoria real del proceso
Pico de tracemalloc	16.410.314	15.65	asignaciones del intérprete
tracemalloc al finalizar	5.078.794	4.84	objetos vivos al cierre

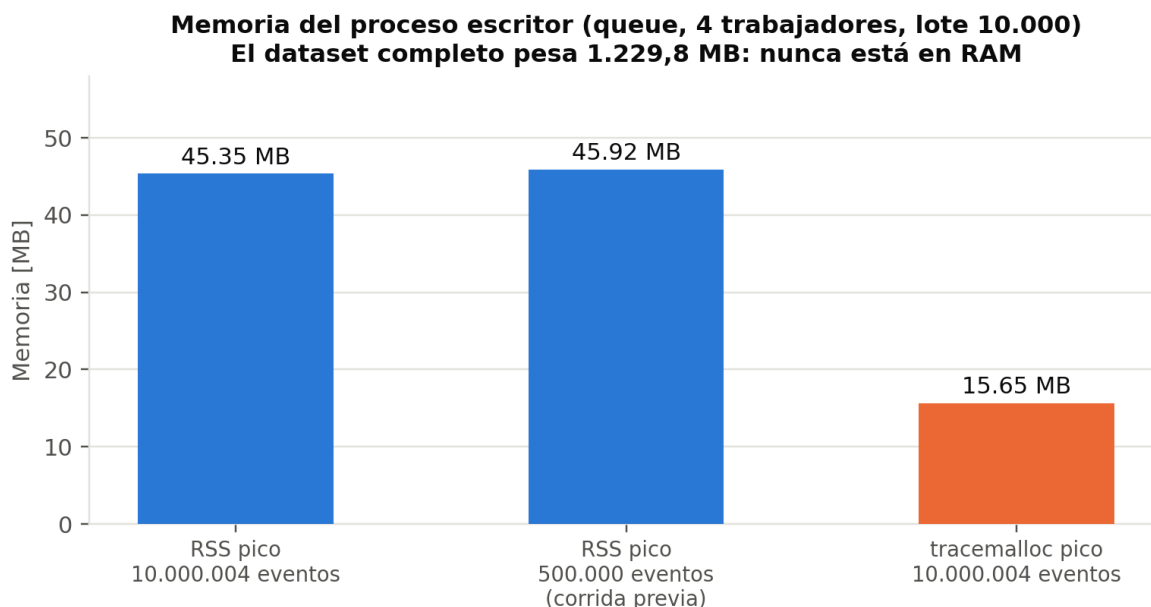


Figura 6. Memoria pico del escritor. La corrida de 500.000 eventos es una medición previa con la misma configuración de cola.

6.1 Por qué maxsize = 64

La cola se declara en `src/arquitectura_a.py:96` como `mp.Queue(maxsize=64)`. El parámetro no es cosmético: es el único mecanismo que impide que los trabajadores, que producen más rápido de lo que el escritor consume, acumulen el dataset completo en RAM. Al alcanzarse el límite, `put()` bloquea al trabajador hasta que el escritor drene un lote —eso es la contrapresión—, con lo que la memoria en tránsito queda acotada por construcción y no por suerte.

El valor 64 sale de acotar la memoria en tránsito sin estrangular el paralelismo. Con lotes de 10.000 eventos y un promedio de 128.95 bytes por evento en disco, 64 lotes en tránsito representan del orden de 79 MB de texto: un techo acotado y predecible, frente a los 1.229,8 MB que ocuparía el dataset entero. Al mismo tiempo, 64 es holgado respecto de los 4 trabajadores del experimento, de modo que un trabajador rara vez debería quedar bloqueado esperando espacio.

El RSS medido, 45.35 MB, queda por debajo de esa cota. Como la cota corresponde a la cola completamente llena, el dato indica que la cola no permaneció saturada durante la corrida, es decir que el escritor consiguió mantener el ritmo de los cuatro trabajadores. Conviene precisar que se trata de una inferencia a partir del pico de memoria y no de una medición directa de la ocupación de la cola, que no se instrumentó.

6.2 RSS y tracemalloc no miden lo mismo

Las dos cifras de la Tabla 9 difieren en casi un factor tres y no deben tratarse como equivalentes. `tracemalloc` solo rastrea las asignaciones que realiza el intérprete de CPython —listas, diccionarios, cadenas, objetos `bytes`— y no ve la memoria reservada por extensiones en C ni los búferes internos del sistema operativo, entre ellos el búfer de escritura de 4 MB con que se abre el archivo. El conjunto residente, en cambio, es lo que el sistema operativo contabiliza como memoria física realmente ocupada por el proceso, y es por tanto la cifra autoritativa: 45.35 MB. El valor de `tracemalloc` es útil para saber qué parte de ese consumo es atribuible a estructuras Python (15.65 MB, aproximadamente un tercio), no para reemplazarlo.

6.3 Desbordamiento y prueba de que la contrapresión funciona

No se observó desbordamiento ni bloqueo permanente durante las pruebas: ninguna corrida se colgó, abortó ni terminó con un conteo de eventos inferior al esperado. Corresponde matizar el alcance de esta afirmación: se sustenta en que todas las corridas completaron correctamente, no en una medición directa de la ocupación de la cola instante a instante, que no se instrumentó.

La evidencia más contundente de que la contrapresión cumple su función es comparativa. Una corrida previa con 500.000 eventos —veinte veces menos datos, misma configuración de cola— tuvo un pico de 45.92 MB, frente a los 45.35 MB de la corrida de 10.000.004 eventos. Multiplicar el volumen por veinte no aumentó la memoria del escritor; de hecho la dejó marginalmente más baja, dentro de la variación normal entre corridas. Ese es exactamente el comportamiento que un diseño con contrapresión debe exhibir: el consumo del escritor depende del maxsize de la cola y del tamaño de lote, no del tamaño total del dataset. Un diseño sin contrapresión habría mostrado un crecimiento aproximadamente proporcional.

7. Interpretación crítica: dónde y por qué se aplana la curva

El aplanamiento no ocurre en un punto único sino de forma progresiva: la ganancia marginal por trabajador cae de 0.448 en el tramo 2 → 4, a 0.290 en el tramo 4 → 8, a 0.134 en el tramo 8 → 16. La eficiencia cruza el 50 % entre 4 y 8 trabajadores. Esta sección evalúa las cuatro causas candidatas y determina cuál sostiene la evidencia propia.

7.1 Saturación de disco: descartada

Dos mediciones propias la descartan, y ninguna depende de la ficha técnica del NVMe (cuyo ancho de banda máximo no se midió).

- **El experimento de pickle es concluyente.** Las cuatro corridas de la Tabla 8 escriben exactamente los mismos datos, con los mismos 4 trabajadores, en el mismo disco. Solo cambia cuántos eventos viajan por cada `put()`. Y sin embargo la corrida con lotes de 1 tarda 55.423 s y la de lotes de 1.000 tarda 39.558 s. Un cuello de botella en el disco no puede producir una diferencia de 40 % ante una carga de escritura idéntica: el factor limitante está aguas arriba, en el camino de CPU.
- **El caudal nunca alcanza una meseta.** El rendimiento de escritura crece de forma monótona en todo el rango, de 13.59 a 68.52 MB/s, y el mayor salto relativo se produce justo en el último tramo medido. Un dispositivo saturado exhibe un techo plano; aquí no aparece ninguno.
- **El número de flujos de escritura es indiferente.** Con 8 trabajadores, `shard` escribe con ocho descriptores concurrentes y alcanza 53.75 MB/s, mientras que `queue` escribe con un único descriptor y alcanza 54.01 MB/s. Si el subsistema de almacenamiento fuera el límite, cambiar de ocho flujos a uno tendría algún efecto medible; no lo tiene.

7.2 SMT que no aporta: parcialmente descartada, y contra lo esperado

El enunciado del proyecto anticipa que en un equipo de 8 núcleos el rendimiento con 16 procesos podría ser peor que con 8, y pide reportarlo tal cual si ocurre. En este equipo no ocurrió, y se reporta igualmente tal cual: 16 trabajadores fueron más rápidos que 8, con una mediana de 18.001 s frente a 22.879 s, es decir un 21.32 % menos de tiempo. Los hilos SMT sí aportaron trabajo útil.

Lo que sí se cumple es la versión cuantitativa de la advertencia. Los 8 núcleos físicos llevaron el speedup de 1.00x a 3.96x, un aporte de 2.96. Los 8 hilos lógicos adicionales lo llevaron de 3.96x a 5.03x, un aporte de 1.07: apenas un 36 % de lo que aportó el hardware real. La explicación coherente con el código es que la carga no es aritmética pura: cada evento implica formateo de timestamps, construcción de diccionarios, serialización JSON y escritura, de modo que los hilos hermanos encuentran huecos de emisión mientras el otro espera memoria o E/S. Esa es una interpretación del mecanismo, no una medición: no se instrumentaron contadores de rendimiento del procesador.

La frontera SMT sí explica por qué el rendimiento marginal se reduce a la mitad precisamente en el tramo 8 → 16: es el único tramo en el que los trabajadores agregados no corresponden a núcleos de ejecución independientes.

7.3 Costo de creación de procesos con spawn: acotada, no dominante

El costo de `spawn` está dentro de la ventana cronometrada, porque los procesos se crean después de `t_inicio` (`src/main.py:104` y `src/main.py:116`). No se instrumentó por separado, de modo que su magnitud exacta es un dato no disponible. Aun así, la propia curva lo acota: es un costo fijo por proceso que crece linealmente con n , mientras que la pérdida de eficiencia ya alcanza el 30 % con solo 4 trabajadores —cuatro creaciones de proceso sobre una corrida de 32 s—. Si `spawn` fuera la causa dominante, la eficiencia debería mantenerse alta con pocos trabajadores y desplomarse solo al aumentar n ; lo que se observa es una degradación desde el comienzo. El código además evita el fallo clásico asociado a `spawn` en Windows —la reimportación recursiva del módulo— mediante la guarda `if __name__ == "__main__"` en todos los puntos de entrada (`src/main.py:197`, `bench/costo_pickle.py:195`, `bench/worker_mem.py:188`).

7.4 Fracción secuencial según la Ley de Amdahl: causa dominante

La Ley de Amdahl establece que si una fracción f del trabajo no es paralelizable, el speedup queda limitado por $S(n) = 1 / (f + (1 - f)/n)$. Despejando f a partir del speedup medido se obtiene la métrica de Karp-Flatt, $f = (n/S(n) - 1) / (n - 1)$, que estima la fracción serial experimentalmente para cada configuración. Su valor diagnóstico no está en la magnitud sino en la tendencia: si f se mantiene constante al aumentar n , la limitación es genuinamente una fracción serial fija; si f crece con n , la limitación proviene de sobrecostos que escalan con el número de procesos, como la comunicación o la creación de procesos.

Tabla 10. Fracción serial estimada por Karp-Flatt sobre las medianas de `shard`, y speedup predicho por el modelo de Amdahl ajustado con $f = 0.1461$ (valor obtenido en $n = 8$).

Trabajadores	$S(n)$ medido	f estimada	$S(n)$ según Amdahl	Ganancia marginal por trabajador
1	1.00x	—	1.00x	—
2	1.90x	0.0531	1.75x	0.899
4	2.80x	0.1435	2.78x	0.448
8	3.96x	0.1461	3.96x	0.290
16	5.03x	0.1455	5.01x	0.134

El resultado es contundente: entre 4, 8 y 16 trabajadores la fracción serial estimada permanece prácticamente constante en 0.1435, 0.1461 y 0.1455. Esa estabilidad es la firma de una fracción serial fija y no de un sobrecosto creciente con n , y es la razón por la cual se identifica a la Ley de Amdahl como causa dominante del aplanamiento. El modelo ajustado con $f = 0.1461$ reproduce lo medido con un error inferior al 1 % en las tres configuraciones superiores, como muestra la curva verde de la Figura 2. Con ese valor, el speedup máximo teóricamente alcanzable en este sistema, aun con infinitos procesos, es $1/f \approx 6.84x$; ya con 16 trabajadores se alcanza 5.03x, es decir el 73 % de ese techo.

Hay una desviación que conviene señalar en lugar de omitir: en $n = 2$ la fracción estimada es 0.0531, muy inferior al resto. Un modelo de una sola f no describe bien el tramo $1 \rightarrow 2$, donde el equipo aún dispone de núcleos ociosos de sobra y prácticamente no hay contención. La fracción serial efectiva, por tanto, no es una constante del programa sino que emerge al aumentar la presión sobre los recursos compartidos.

Corresponde además ser preciso sobre lo que la métrica de Karp-Flatt puede y no puede afirmar: cuantifica cuánta fracción serial hay, pero no identifica cuál es. Los candidatos plausibles en este código son la creación e importación de procesos, la apertura y cierre de archivos por trabajador, y sobre todo la contención por ancho de banda de memoria y por el camino de escritura del sistema operativo, que matemáticamente se manifiesta igual que una fracción serial. Determinar cuál pesa más exigiría perfilado por proceso, que no se realizó: es un dato no disponible.

7.5 Comparación de arquitecturas y decisión de producción

Las dos arquitecturas rinden de forma muy similar, y la comparación honesta reparte los puntos. Shard es más rápida con 1, 2 y 16 trabajadores; queue es marginalmente más rápida con 4 y 8. La diferencia más notoria aparece en 16 trabajadores, donde shard alcanza 18.001 s frente a 18.926 s de queue.

En contra de shard hay un dato que también corresponde reconocer: en el tramo de 8 a 16 trabajadores, queue sostiene un poco mejor la eficiencia relativa, y en 4 y 8 trabajadores obtiene eficiencias superiores (74.9 % y 51.4 % frente a 69.9 % y 49.4 %). Es decir, la ventaja de shard no es uniforme a lo largo de la curva.

La arquitectura definitiva elegida para el dataset de producción es shard, y la justificación no descansa solo en la velocidad. Shard no requiere coordinación entre procesos, cola compartida ni proceso escritor único, lo que elimina el modo de falla que se analiza en la defensa oral —si el escritor único muere, los trabajadores se bloquean sobre una cola llena— y reduce la complejidad operativa. Suma a eso que es igual o más rápida en tres de las cinco configuraciones medidas, incluida la más veloz de todas.

7.6 Punto dulce y configuración del dataset final

Aplicando como criterio la última configuración cuya eficiencia se mantiene en 50 % o más, el punto dulce de la arquitectura shard es 4 trabajadores, con 69.9 % de eficiencia; la configuración siguiente ya cae a 49.4 %. El dataset final de producción se generó precisamente con 4 trabajadores, de modo que la elección de producción coincide con el punto dulce identificado en la curva.

Tabla 11. Dataset final de producción (data/raw/manifiesto.json).

Parámetro	Valor
Eventos solicitados	10.000.004
Eventos generados	10.000.004
Bytes totales	1.289.499.073
Arquitectura	shard
Trabajadores	4
Semilla	42
Duración	28.289 s
Archivos	part-000.jsonl a part-003.jsonl, 2.500.001 eventos cada uno

Un último dato cierra el argumento de reproducibilidad. El total de bytes del dataset final, 1.289.499.073, es idéntico byte por byte al de la fila de 4 trabajadores del benchmark de la sección 2, pese a tratarse de corridas independientes separadas en el tiempo. Con la misma semilla y el mismo número de trabajadores, el generador produce datasets idénticos: la concurrencia no introdujo ninguna indeterminación en el resultado.

Se advierte, no obstante, que la duración del dataset final (28.289 s) no es comparable con la mediana de 4 trabajadores del benchmark (32.362 s): son corridas independientes, la primera sin corrida de calentamiento previa ni repeticiones, por lo que no debe leerse como una mejora sino como una única observación adicional dentro del rango ya conocido de dispersión de esa configuración (30.643 s a 33.077 s).